```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords

# Download stopwords if not already downloaded (quietly)
try:
    nltk.data.find('corpora/stopwords')
except nltk.downloader.DownloadError:
    nltk.download('stopwords', quiet=True)

# 1. Load the movie review dataset
df = pd.read_csv('movie_reviews.csv')

print("First 5 rows of the dataset:")
print(df.head())

print("\nDataFrame Information:")
df.info()

# 2. Define the preprocess_text function
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove non-alphabetic characters and extra spaces
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize
    words = text.split()
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    words = [word for word in words if word not in stop_words]
    # Join back into a string
    return ' '.join(words)

print("\nPreprocessing function defined.")
```

```
First 5 rows of the dataset:
                                              text sentiment
0  This movie was absolutely fantastic! I loved e...  positive
1  Terrible film, a complete waste of time. Avoid...  negative
2  It was an okay movie, nothing special but not ...   neutral
3  The acting was superb and the plot was engagin...  positive
4  I hated the ending, it made no sense and ruine...  negative

DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   text       5 non-null      object
 1   sentiment  5 non-null      object
dtypes: object(2)
memory usage: 212.0+ bytes

Preprocessing function defined.
```

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords

# Download stopwords if not already downloaded (quietly)
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
```

```
    nltk.download('stopwords', quiet=True)

# 1. Load the movie review dataset
df = pd.read_csv('movie_reviews.csv')

print("First 5 rows of the dataset:")
print(df.head())

print("\nDataFrame Information:")
df.info()

# 2. Define the preprocess_text function
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove non-alphabetic characters and extra spaces
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize
    words = text.split()
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    words = [word for word in words if word not in stop_words]
    # Join back into a string
    return ' '.join(words)

print("\nPreprocessing function defined.")
```

```
First 5 rows of the dataset:
                                                text sentiment
0  This movie was absolutely fantastic! I loved e...  positive
1  Terrible film, a complete waste of time. Avoid...  negative
2  It was an okay movie, nothing special but not ...   neutral
3  The acting was superb and the plot was engagin...  positive
4  I hated the ending, it made no sense and ruine...  negative

DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   text       5 non-null      object
 1   sentiment  5 non-null      object
dtypes: object(2)
memory usage: 212.0+ bytes

Preprocessing function defined.
```

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords

# Create a dummy movie_reviews.csv file for demonstration
# In a real scenario, this file would already exist or be provided.
dummy_data = {
    'text': [
        'This movie was absolutely fantastic! I loved every minute of it.',
        'Terrible film, a complete waste of time. Avoid at all costs.',
        'It was an okay movie, nothing special but not bad either.',
        'The acting was superb and the plot was engaging. Highly recommend.',
        'I hated the ending, it made no sense and ruined the whole experience.'
    ],
    'sentiment': ['positive', 'negative', 'neutral', 'positive', 'negative']
}
dummy_df = pd.DataFrame(dummy_data)
dummy_df.to_csv('movie_reviews.csv', index=False)
print("Dummy 'movie_reviews.csv' created.")

# Download stopwords if not already downloaded (quietly)
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords', quiet=True)

# 1. Load the movie review dataset
df = pd.read_csv('movie_reviews.csv')

print("First 5 rows of the dataset:")
```

```
print("First 5 rows of the dataset:")
print(df.head())

print("\nDataFrame Information:")
df.info()

# 2. Define the preprocess_text function
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove non-alphabetic characters and extra spaces
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize
    words = text.split()
    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    words = [word for word in words if word not in stop_words]
    # Join back into a string
    return ' '.join(words)

print("\nPreprocessing function defined.")
```

```
Dummy 'movie_reviews.csv' created.
First 5 rows of the dataset:
                                                text sentiment
0  This movie was absolutely fantastic! I loved e...  positive
1  Terrible film, a complete waste of time. Avoid...  negative
2  It was an okay movie, nothing special but not ...   neutral
3  The acting was superb and the plot was engagin...  positive
4  I hated the ending, it made no sense and ruine...  negative

DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   text       5 non-null      object
 1   sentiment  5 non-null      object
dtypes: object(2)
memory usage: 212.0+ bytes

Preprocessing function defined.
```

```
df['cleaned_text'] = df['text'].apply(preprocess_text)

print("First 5 rows with original and cleaned text:")
print(df[['text', 'cleaned_text']].head())
```

```
First 5 rows with original and cleaned text:
                                                text  \
0  This movie was absolutely fantastic! I loved e...
1  Terrible film, a complete waste of time. Avoid...
2  It was an okay movie, nothing special but not ...
3  The acting was superb and the plot was engagin...
4  I hated the ending, it made no sense and ruine...

                                       cleaned_text
0    movie absolutely fantastic loved every minute
1    terrible film complete waste time avoid costs
2            okay movie nothing special bad either
3     acting superb plot engaging highly recommend
4  hated ending made sense ruined whole experience
```

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['cleaned_text'])

vocabulary_size = len(vectorizer.vocabulary_)
print(f"Size of the TF-IDF vocabulary: {vocabulary_size}")

print("TF-IDF features successfully created.")
```

```
Size of the TF-IDF vocabulary: 31
TF-IDF features successfully created.
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Get the target variable
```

```
y = df['sentiment']

# 1. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

# 2. Instantiate a Logistic Regression model
# Set max_iter to a suitable value to ensure convergence
# Set random_state for reproducibility
model = LogisticRegression(max_iter=1000, random_state=42)

# 3. Train the Logistic Regression model
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")
```

```
X_train shape: (4, 31)
X_test shape: (1, 31)
y_train shape: (4,)
y_test shape: (1,)
Logistic Regression model trained successfully.
```

```
import warnings
warnings.filterwarnings('ignore')

# 1. Use the trained model to make predictions on the test set
y_pred = model.predict(X_test)

print("Model predictions on test set generated.")

# 2. Calculate and print Accuracy Score
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.4f}")

# 3. Calculate and print Precision, Recall, and F1-score
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

print(f"Precision (weighted): {precision:.4f}")
print(f"Recall (weighted): {recall:.4f}")
print(f"F1-Score (weighted): {f1:.4f}")

# 4. Generate and print Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

```
Model predictions on test set generated.

Accuracy: 0.0000
Precision (weighted): 0.0000
Recall (weighted): 0.0000
F1-Score (weighted): 0.0000

Confusion Matrix:
[[0 1]
 [0 0]]
```